7-1 Final Project

Design Decision for the Three-Dimensional Desk Scene

Michael Soto

Southern New Hampshire University

CS-330: Computational Graphics and Visualization

Dr. Fernando Nieto

October 18, 2026

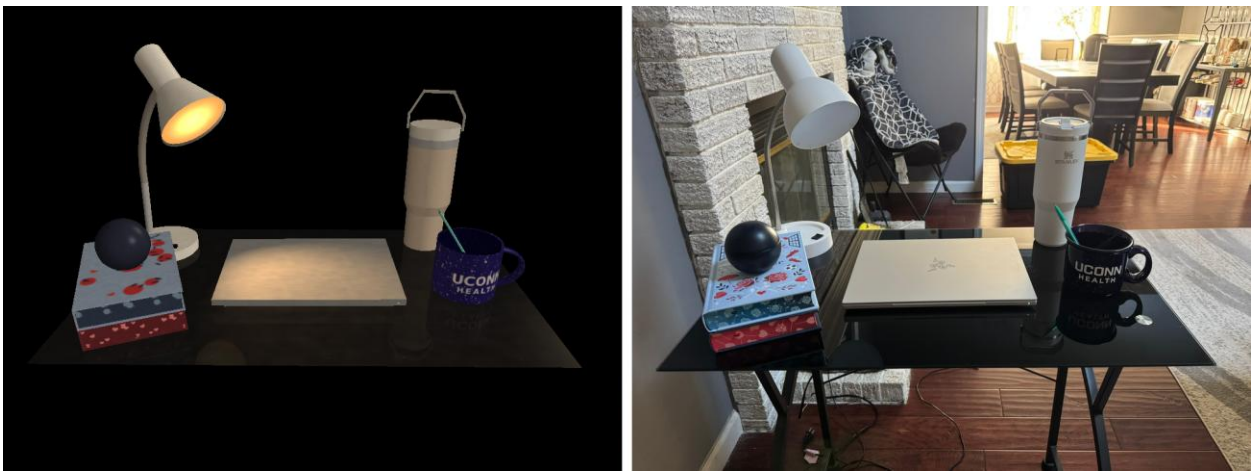


Figure 1. The desk as the program draws it from its starting camera (left) and in the reference photograph IMG_0055 (right), at the same scale and field of view. Photograph by the author.

Development Choices

The scene is the glass desk from my Milestone One proposal, with all five objects kept: the laptop, books and ball tin, tumbler, mug and pen, and gooseneck lamp. I chose them for the range of shapes they need, and with the glass plane they use six of the course's basic shapes. I placed them by measurement, solving each camera from the desk's traces edges in three photographs and fitting each object to its outline, with the mug, 4 units across, as the scale. Once the photograph's 0.91 degree roll is removed, the launch view puts the laptop's edge and center within 2 pixels of the photograph's, and the books', tumbler's, and mug's within 9 (Figure 1). The lamp's right edge misses by 12, 10.4 of them from moving its shade 0.70 units to meet the measured gooseneck.

Every object stays under 1,000 triangles, the mug and pen largest at 956. The course half torus alone draws 1,050 triangles, so my BentTubeMesh class builds the handle's arc in 384. All 16 textures were made for this project at 1024 pixels or more per side. Lighting uses all three

Phong terms (ambient, diffuse, and specular) and four lights. Two directional window lights and a wall-bounce point light take colors measured from the photographs, and a lamp spotlight takes the 2700 K color of a blackbody table (Charity, 2001), dimmed by LearnOpenGL's attenuation for a 32 unit range (de Vries, n.d.).

Table 1 compares the approaches I considered, including four that failed at first. The shade's own cone lit the desk's middle with no visible edge, and Milestone Five's lights, set for the mug alone, left up to 56 percent of the laptop's pixels with a color channel at 255. The course meshes' texture coordinates stretched or skewed images, and mirrored surfaces touching the glass left bright specks where the depth buffer could not separate them.

Table 1. Approaches considered and chosen during development, with what was measured or computed for each

Aspect	Approach considered	Approach chosen	Measured result
Lamp beam	The cone set by the shade's geometry, 27.58 to 51.27 degrees, lamp diffuse 9 (window lights 0.40 and 0.27)	A narrower spot, full strength within 12 degrees and none past 28, lamp diffuse 19 (window lights 0.34 and 0.23); a choice, since the lamp is off in every photograph	Laptop's top face at launch (27,718 pixels): 1,164 pixels take 40 percent or more of their light from the lamp with the wide cone, 11,728 with the spot. With the wide cone, nine tenths of the face take at least 0.19 of their light from the lamp; with the spot, a tenth takes 0.04 or less, so only the spot shows an edge.
Lights	Milestone Five's four lights, set for the mug alone, moved into the full layout	Four re-tuned lights: the 2700 K lamp spotlight, two directional window lights, and a point light for the wall's bounce	Over 22 test views: pixels with a color channel at 255 on the worst object, 56.05 percent before and 0.17 percent after (limit 0.5 percent). On the darkest object, 98 percent of pixels kept at least 0.117 of their unshaded brightness before and 0.262 after (floor 0.25). After: lamp light on white surfaces has green at 0.6627 to 0.6628 and blue at 0.3412 of its red (target 0.6627 and 0.3412).

Aspect	Approach considered	Approach chosen	Measured result
Orthographic view (O key)	Milestone Five's view, which moved the camera to a 6.6 unit tall, level front view of the mug; or sizing the view by where the view ray meets the desk	The camera keeps its place and orientation; the view's half height is the depth of the desk's middle times tan(FOV / 2)	Computed from the code and checked in the running program: at launch the desk's middle lies 30.34 units ahead of the camera, so O opens a view 33.36 units tall, and the program printed 33.3605. The view ray meets the desk 36.28 units away, which would draw the desk's middle 16 percent smaller.
Speed (scroll wheel, Left Shift)	The course camera's step, used through Milestone Five: 1 unit per second added or taken away per notch, 0.1 to 45	Speed times 1.25 per notch, 0.5 to 40, starting at 7; Left Shift moves at 0.25 of the speed	Computed from the code: each notch multiplies or divides the speed by 1.25 until it reaches a limit, while the course's 1 unit step equaled the whole speed at 1 unit per second and 2.2 percent of it at 45. From the start, 12 notches down reach 0.5 and 8 up reach 40. In the running program, 40 wheel notches printed the 40 speeds this rule predicts, and Left Shift cut the camera's travel on screen to 0.25 of normal.
Mug lettering	Milestone Five's method: the print drawn over the glaze in a second pass	One draw: the shader mixes the print texture over the glaze texture by the print's transparency	Measured when I made this change, with Milestone Five's lights on both versions: no pixel differs by more than 2 levels of 255, and the 1,056 to 1,413 pixels that differ in each of 3 views lie on the lettering and its reflection. The two methods are equal in exact arithmetic.
Mug handle	The course half torus: 1,050 triangles, with irregular normals and texture coordinates	BentTubeMesh: a 180 degree arc of 16 by 12 segments, 384 triangles with exact normals	Mug and pen 956 triangles, the scene's largest object (limit 1,000); the half torus alone would pass the limit. The torus streaked the glaze, while the arc shows clean flecks that match the body.
Lamp mouth	A flat disc in the lamp color, RGB (255, 169, 87)	A glow that eases (smoothstep) from (255, 255, 174) at the middle to (255, 169, 87) at the rim	Launch view, 4,633 disc pixels: mean (255, 169, 87) flat and (255, 213, 114) with the glow; 26 percent of the disc reaches full red and green.
Texture mapping of caps, tapered sides, and the ball tin	The course meshes' stored texture coordinates, which stretch or skew the image: cap values typed with errors up to 0.02, and a sinusoidal layout on the sphere	Coordinates computed in the shader from the surface position and chosen per draw by SetSurfaceMapping(): exact planar caps, the tapered side unrolled as a cone, longitude and latitude on the sphere	Share of on-screen texels (texture pixels) that are square, with sides within 10 percent and corners within 6 degrees of square, over 22 views, before and after: lamp 0.606 and 0.987, tumbler 0.907 and 0.965, tin 0.239 and 0.518; caps and tapered sides 1.000 after. The tin's other texels narrow toward its poles as this layout predicts (measured over predicted, median 1.004).

Aspect	Approach considered	Approach chosen	Measured result
Reflection in the glass	Mirrored objects drawn under the glass with the depth test alone, although surfaces touching the glass mirror to within 0.0004 units of it, closer than the depth buffer resolves at about 30 units	The shader discards mirrored fragments less than 0.001 units below the glass	Over 22 views: 10 bright specks (26 to 129 levels above the glass around them) before and 0 after; 0 glass pixels lost; 0 pixels of reflection outside the glass before or after.

Navigation

The console lists the controls at launch, since the captured cursor is hidden. W, A, S, and D move the camera forward, left, back, and right, and Q and E move it up and down along its own up direction. The mouse turns it 0.1 degree per pixel, and the tilt stops at 89 degrees, so the view never flips. The scroll wheel sets the speed, starting at 7 units per second, and holding Left Shift drops it to a quarter. O and P switch between orthographic and perspective views from the same place, and Esc closes the window.

ProcessMovementKeys() scaled each move by the seconds since the last frame, so speed does not depend on frame rate, and Left Shit multiplies that time by PRECISION_FACTOR (0.25). Mouse_Scroll_Wheel_Callback() multiplies the speed by SPEED_STEP (1.25) per notch within 0.5 to 40, so every notch changes the speed by the same factor. SetOrthographicView() keeps the camera in place and, with the desk’s middle at least 0.91 units ahead, sets the view’s half height to that depth time tan(FOV / 2), so objects at that depth keep their size; in that view, W and S zoom. KeepCameraInBounce() keeps the camera at least 0.25 units above the glass and within 80 units of the scene’s center, so the desk and its objects never reach past the 100 unit far plane.

Custom Functions

I added 46 functions to the course's SceneManager and ViewManager classes, plus BentTubeMesh, a startup function, and seven shader functions; Table 2 groups them by purpose. SetObjectPlacement() stores one matrix that moves every part drawn after it, so builders such as RenderDeskLamp() work in their own coordinates and can stand anywhere on the desk. RenderDeskObjects() draws every object twice per frame, once mirrored for RenderDeskReflection() and once upright, so the reflection reuses the same drawing code.

Table 2. The custom functions, grouped by purpose

Purpose	Functions	What they do and how they are reused
Object builders	RenderLaptop(), RenderBookStack(), RenderBook(), RenderBallTin(), RenderTumbler(), RenderTumblerHandle(), RenderDeskLamp(), RenderMugAndPen(), RenderMugBody(), RenderMugInterior(), RenderMugHandle(), RenderPen()	Each draws one object or part, mostly from named constants kept at the top of SceneManager.cpp. RenderBook() takes a BOOK_SPEC, so one function draws both books.
Placement and transforms	PlacementMatrix(), SetObjectPlacement(), ClearObjectPlacement(), SetModelMatrix(), SetSegmentTransformations(), RotateOffset(), DrawBox()	PlacementMatrix(), SetObjectPlacement(), and ClearObjectPlacement() place a whole object. SetModelMatrix(), which every transform helper calls, mirrors shapes for the reflection and sets the normal matrix. SetSegmentTransformations() stands a round mesh along a segment for 5 parts, RotateOffset() keeps the pen tip on its barrel, and DrawBox() draws 9 of the laptop's and books' 12 boxes with texture repeats set by their size.
Shader settings	SetShaderLighting(), SetShaderOverlay(), ClearShaderOverlay(), SetBoxProjection(), ClearBoxProjection(), SetCylinderUVScale(), SetShaderNormals(), SetSurfaceMapping()	Each switches one shader feature for the shapes drawn next: unlit glow, the lettering overlay, box projection, repeats that keep a cylinder's texels square, exact normals, and the cap, taper, and sphere mappings. Set and clear pairs keep a feature from carrying into later shapes.
Scene setup and lights	LoadSceneTextures(), DefineObjectMaterials(), SetupSceneLights(), ApplySceneLights(), SetShaderLight()	The course tutorials name the first three, and PrepareScene() calls them once; builders then look textures and materials up by tag. ApplySceneLights() sends the four lights through SetShaderLight(), mirrored below the glass while the reflection is drawn.

Purpose	Functions	What they do and how they are reused
Desk and reflection	RenderDeskReflection(), RenderDeskSurface(), DrawDeskPlane(), RenderDeskObjects()	RenderDeskReflection() uses the stencil buffer to keep the mirror image to the visible glass, and RenderDeskSurface() blends the glass over it. DrawDeskPlane() serves both the stencil pass and the glass, so they cover the same pixels.
Camera input	Mouse_Scroll_Wheel_Callback(), Window_Resize_Callback(), ProcessProjectionKeys(), ProcessMovementKeys(), KeepCameraInBounds(), PrintControls()	The course's ProcessKeyboardEvents() handled only Esc; it now hands the projection and movement keys to their own functions, and every move ends with the bounds check. The wheel and resize callbacks use the course tutorial's names; the first sets the speed, and the second keeps the aspect ratio and skips a minimized window's zero size. PrintControls() lists the controls.
Projection	SetOrthographicView(), SetPerspectiveView(), GetPerspectiveProjection(), GetOrthographicProjection()	SetOrthographicView() and SetPerspectiveView() switch views without moving the camera; the two Get functions build each matrix from the window's current aspect ratio with the shared 0.1 and 100 near and far planes.
Mesh and startup	BentTubeMesh: Load(), Draw(), TriangleCount(); SetWorkingDirectoryToProject() in MainCode.cpp	One class builds both the 180 degree mug handle and the 49.6 degree gooseneck, and TriangleCount() returns how many triangles Draw() sends. The startup function makes the project folder the working directory, so the relative Shaders and Textures paths resolve when the EXE is started directly.
Fragment shader	AddLightSource(), SurfaceNormal(), SurfaceCoordinate(), UnrolledCoordinate(), SphereCoordinate(), SeamGradients(), SampleTexture()	AddLightSource(), which replaces the course's CalcLightSource(), adds one light's ambient, diffuse, and specular terms; SurfaceNormal() rebuilds exact normals for the course cylinder, cone, and tapered cylinder; and the other five make and sample texture coordinates for each kind of surface, blending the taper's seam.

BentTubeMesh needs only GLEW, GLM, and the course meshes' vertex layout, so any course project could add its two files to bend pipes or rings. The white plastic cylinder with a mapped flat end should have been a function, since it is written out three times, for the tumbler's lid, the lamp's base, and the shade's neck, and all three repeat the same ten shader and draw calls. A DrawCappedCylinder() helper would hold those calls once and always switch the cap mapping back off.

References

Charity, M. (2001). *Blackbody color datafile* (Version 2001-Jun-22) [Data set]. vendian.org. http://www.vendian.org/mncharity/dir3/blackbody/UnstableURLs/bbr_color.html

de Vries, J. (n.d.). *Light casters*. LearnOpenGL. https://learnopengl.com/Lighting/Light-casters